

Student Information System

Design Patterns — Analysis, Diagrams & Justification

Software Modeling and Design Course

github.com/eni6667/Student-Information-System

01

Singleton Pattern

Creational — ReportManager

02

Facade Pattern

Structural — StudentSystemFacade

03

Observer Pattern

Behavioral — Grade Notification

Introduction — Design Patterns in the SIS

Design patterns are proven, reusable solutions to recurring problems in object-oriented design, formalized by the Gang of Four (GoF). They are divided into three categories: **Creational** (how objects are created), **Structural** (how classes and objects are composed), and **Behavioral** (how objects communicate and share responsibility).

The Student Information System manages three actor types — **Student**, **Teacher**, and **Admin** — and implements features including user registration and login, course enrollment and management, grade assignment and tracking, student/teacher profile management, and report generation (grade reports, course reports, and financial/administrative reports). Each of the three patterns below is directly grounded in the system's use cases, swimlane diagrams, activity diagrams, and ERD present in the project repository.

#	Pattern	Category	Applies to in SIS	GoF Intent
01	Singleton	Creational	ReportManager — single engine for all grade, course & financial reports	Ensure a class has only one instance and provide global access
02	Facade	Structural	StudentSystemFacade — unified interface over Auth, Course, Grade & Report services	Provide a simplified interface to a complex subsystem
03	Observer	Behavioral	Grade notifies Student & Admin when a grade is assigned or updated	Define a one-to-many dependency so dependents are notified automatically

01

Singleton Pattern

Creational Pattern

Intent: Ensure a class has only one instance and provide a global point of access to it.

What is the Singleton Pattern?

The Singleton pattern restricts instantiation of a class to a single object. It stores that instance as a private static field, blocks public construction, and exposes a static *getInstance()* method that always returns the same object. This guarantees that no matter how many times the instance is requested across the system, only one will ever exist — sharing state, configuration, and data consistently.

Where is it present in the SIS?

The SIS generates several types of reports visible across the project's activity diagram (Generate Report) and use cases: grade reports for students, course performance reports for teachers, and financial/administrative reports for admins. A **ReportManager** singleton manages all report creation and distribution. It holds the list of generated reports, handles export, and is the single authority for all reporting operations in the system.

- Admin calls *ReportManager.getInstance().generateFinancialReport()* from the Generate Report activity
- Student calls *getInstance().generateGradeReport(student)* to download their transcript
- Teacher accesses *getInstance().generateCourseReport(course)* to see class performance

Justification

If *ReportManager* were not a Singleton, each actor could create their own instance, leading to duplicate report data, inconsistent financial totals, and race conditions when multiple admins generate reports simultaneously. A single shared instance ensures all actors see the same report state. This directly supports the **Single Responsibility Principle** — one class, one report engine — and **Dependency Inversion** — actors depend on the *ReportManager* abstraction, not on individual report-creation logic scattered across the system.

UML Structure Diagram

Singleton: ReportManager ensures a single shared report engine

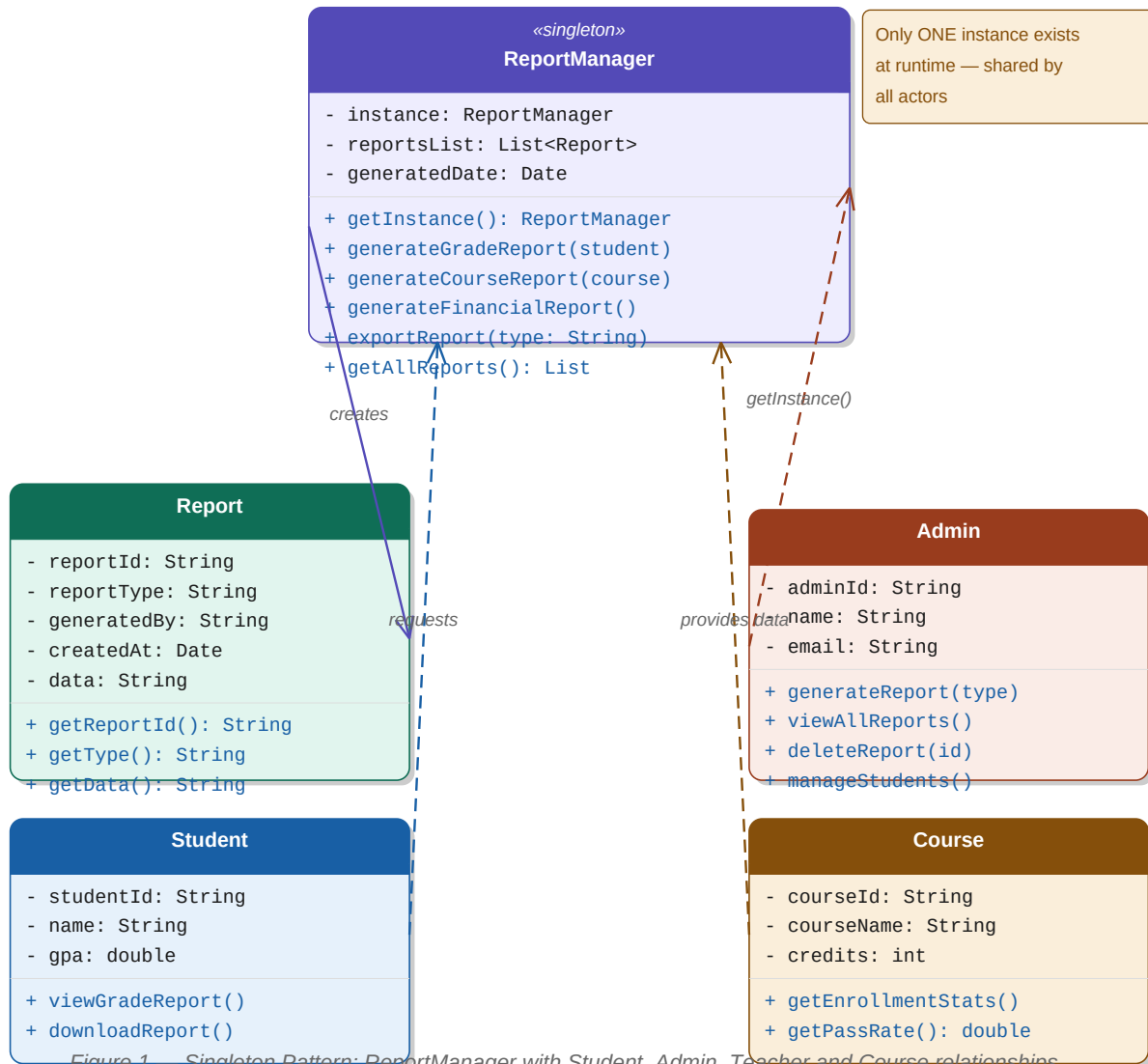


Figure 1 — Singleton Pattern: ReportManager with Student, Admin, Teacher and Course relationships

Java Implementation

```

public class ReportManager {
    // Single static instance – only one will ever exist
    private static ReportManager instance;
    private List reportsList = new ArrayList<>();
    private Date generatedDate;

    // Private constructor – prevents external instantiation
    private ReportManager() {}

    // Global access point
    public static synchronized ReportManager getInstance() {
        if (instance == null) {
            instance = new ReportManager();
        }
        return instance;
    }

    public Report generateGradeReport(Student student) {
        Report r = new Report("GRADE", student.getName(), new Date());
    }
}
  
```

```
reportsList.add(r);
return r;
}
public Report generateCourseReport(Course course) {
    Report r = new Report("COURSE", course.getCourseName(), new Date());
    reportsList.add(r);
    return r;
}
public Report generateFinancialReport() {
    Report r = new Report("FINANCIAL", "Admin", new Date());
    reportsList.add(r);
    return r;
}
public void exportReport(String type) { /* PDF / CSV export logic */ }
public List getAllReports() { return reportsList; }
}

// Usage – anywhere in the system
ReportManager rm = ReportManager.getInstance();
rm.generateGradeReport(student); // same instance every time
```

02

Facade Pattern

Structural Pattern

Intent: Provide a simplified, unified interface to a set of interfaces in a subsystem.

What is the Facade Pattern?

The Facade pattern wraps a complex set of subsystems behind a single, simple interface. Client code interacts only with the facade — it never needs to know which subsystem handles which operation. The facade delegates internally to the appropriate service. This reduces coupling, simplifies client code, and makes the system easier to maintain and refactor without touching the clients.

Where is it present in the SIS?

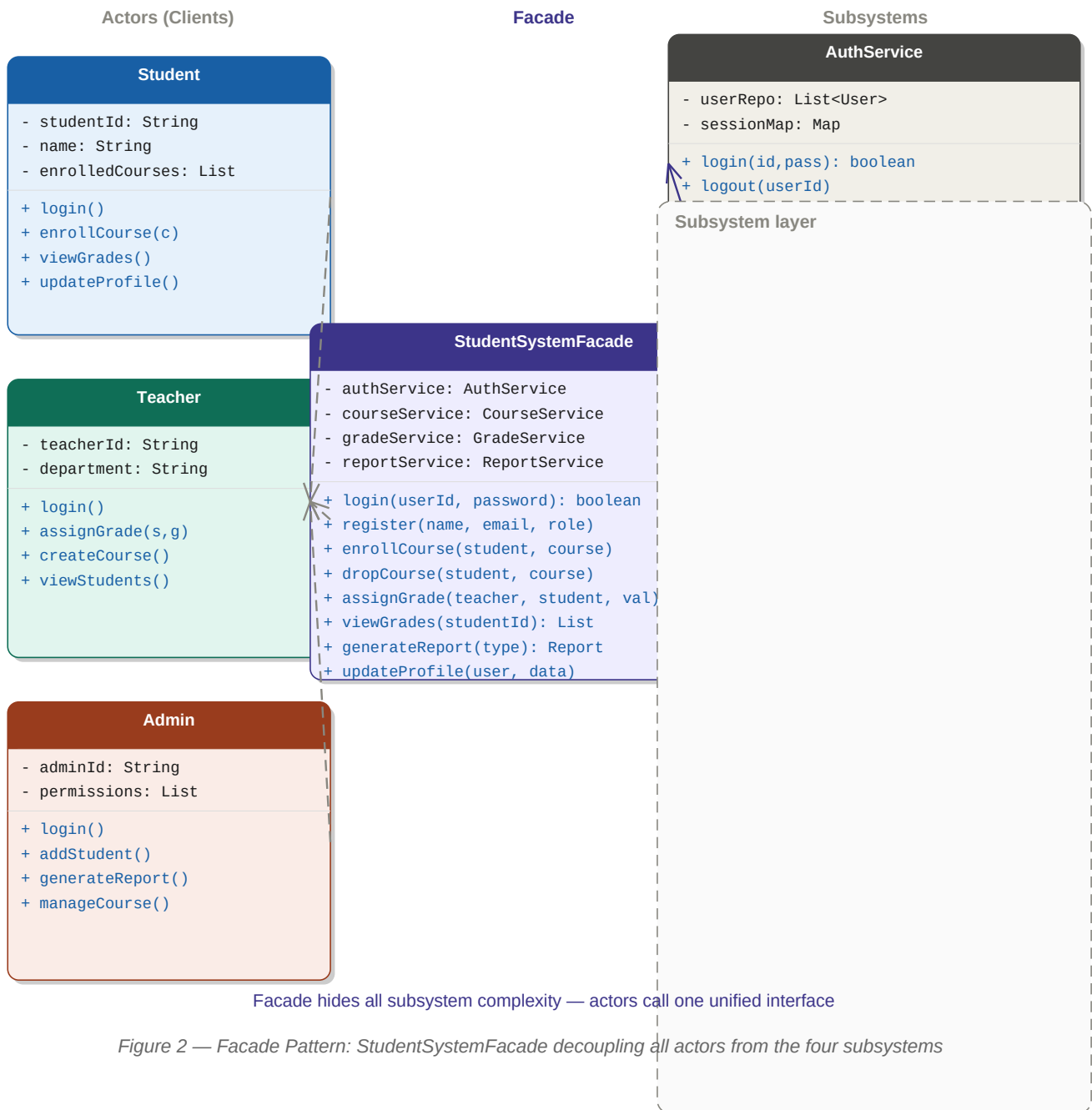
The SIS has four distinct subsystems — authentication, course management, grade handling, and report generation — each with its own logic and data. Without a Facade, every actor (Student, Teacher, Admin) would need to import and directly call methods on AuthService, CourseService, GradeService, and ReportService. A **StudentSystemFacade** provides a single unified interface covering all operations across the system's use cases:

- login/register → delegates to AuthService
- enrollCourse / dropCourse → delegates to CourseService
- assignGrade / viewGrades → delegates to GradeService
- generateReport / exportReport → delegates to ReportService (via Singleton ReportManager)
- updateProfile → delegates to AuthService

Justification

The swimlane diagrams (Register, Enroll, Assign Grade, Add Student) each show the actor interacting with a single 'System' layer before anything reaches the data store. The Facade is exactly that system layer. Without it, a Student UI would need references to CourseService, GradeService, and ReportService all at once — a fragile design. The Facade enforces the **Law of Demeter**: each actor talks only to its immediate collaborator (the Facade). It also implements the **Open/Closed Principle**: a new subsystem (e.g. NotificationService) can be added without modifying any actor class.

UML Structure Diagram



Java Implementation

```
public class StudentSystemFacade {
    // Internal subsystem references – hidden from clients
    private final AuthService authService = new AuthService();
    private final CourseService courseService = new CourseService();
    private final GradeService gradeService = new GradeService();
    private final ReportService reportService = new ReportService();

    // === Authentication ===
    public boolean login(String userId, String password) {
        return authService.login(userId, password);
    }
    public void register(String name, String email, String role) {
        authService.register(name, email, role);
    }
}
```

```

// === Course Management ===
public void enrollCourse(Student student, Course course) {
    courseService.enroll(student, course);
}
public void dropCourse(Student student, Course course) {
    courseService.drop(student, course);
}
public List listAvailableCourses() {
    return courseService.listCourses();
}

// === Grade Management ===
public void assignGrade(Teacher teacher, Student student, double value) {
    gradeService.assignGrade(teacher, student, value);
}
public List viewGrades(String studentId) {
    return gradeService.viewGrades(studentId);
}

// === Report Generation (uses Singleton ReportManager internally) ===
public Report generateReport(String type) {
    return reportService.generate(type);
}
public void updateProfile(User user, Map data) {
    authService.updateProfile(user, data);
}

// Usage – Student actor sees one clean interface, nothing else:
StudentSystemFacade sis = new StudentSystemFacade();
sis.login("s001", "pass123");
sis.enrollCourse(student, course);
sis.viewGrades("s001");

```


03

Observer Pattern

Behavioral Pattern

Intent: Define a one-to-many dependency so all dependents are notified when one object changes state.

What is the Observer Pattern?

The Observer pattern defines a Subject (observable) that maintains a list of Observers and notifies them automatically whenever its state changes. Observers register with the subject; the subject never needs to know the concrete types of its observers, only that they implement the Observer interface. This creates a loosely coupled publish-subscribe mechanism.

Where is it present in the SIS?

The SIS's **Assign Grade swimlane diagram** shows a flow where the Teacher submits a grade, the System processes it, and the Student is notified. The Observer pattern is exactly the mechanism behind this. The **Grade** class is the Subject: when *assign()* or *update()* is called, it calls *notifyObservers()*. Two concrete observers are registered:

- **Student** — receives the notification, updates their grade list, recalculates GPA, and can download their transcript
- **Admin** — receives the notification, logs it to the audit trail, flags anomalies, and can trigger report pre-computation

The Teacher is not an observer — it is the actor that triggers the state change. The Course is the context in which the grade exists, providing enrollment and pass-rate data.

Justification

Without the Observer pattern, `Teacher.assignGrade()` would contain direct calls to `student.notify()` and `admin.notify()`, tightly coupling three unrelated classes. Adding a parent portal notification would require modifying the Teacher class — violating the **Open/Closed Principle**. It also violates **Single Responsibility**: the Teacher's job is to assign grades, not to manage who gets notified and how. With the Observer pattern, Grade fires one event; any registered party reacts independently. This matches the swimlane's separation of concerns exactly: the grade update and the student notification are shown as distinct flow steps.

UML Structure Diagram

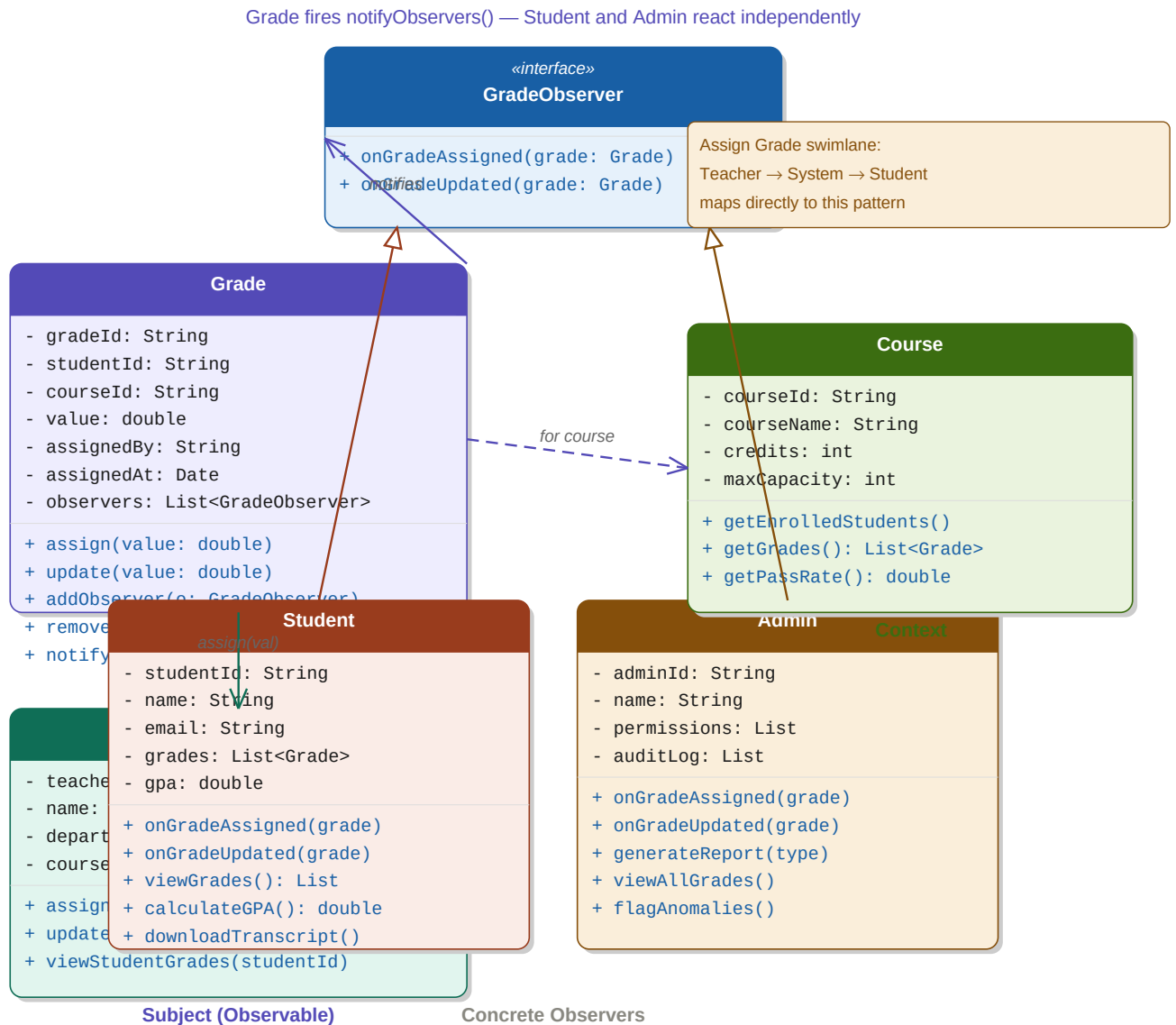


Figure 3 — Observer Pattern: Grade (Subject) notifying Student and Admin (Observers) on assignment

Java Implementation

```

// Observer interface
public interface GradeObserver {
    void onGradeAssigned(Grade grade);
    void onGradeUpdated(Grade grade);
}

// Subject (Observable)
public class Grade {
    private String gradeId, studentId, courseId, assignedBy;
    private double value;
    private Date assignedAt;
    private List observers = new ArrayList<>();

    public void addObserver(GradeObserver o) { observers.add(o); }
    public void removeObserver(GradeObserver o) { observers.remove(o); }
}
  
```

```

// Assign a new grade – triggers all observers
public void assign(double value, String teacherId) {
    this.value = value;
    this.assignedBy = teacherId;
    this.assignedAt = new Date();
    for (GradeObserver o : observers) o.onGradeAssigned(this);
}

// Update existing grade – triggers all observers
public void update(double newValue) {
    this.value = newValue;
    for (GradeObserver o : observers) o.onGradeUpdated(this);
}
}

// Concrete Observer 1: Student
public class Student implements GradeObserver {
    private List grades = new ArrayList<>();
    private double gpa;

    public void onGradeAssigned(Grade g) {
        grades.add(g);
        this.gpa = calculateGPA();
        System.out.println("Notification: grade assigned – " + g.getValue());
    }
    public void onGradeUpdated(Grade g) {
        this.gpa = calculateGPA();
    }
}

// Concrete Observer 2: Admin
public class Admin implements GradeObserver {
    private List auditLog = new ArrayList<>();

    public void onGradeAssigned(Grade g) {
        auditLog.add("Grade assigned: " + g.getGradeId() + " = " + g.getValue());
        flagAnomalies(g);
    }
    public void onGradeUpdated(Grade g) {
        auditLog.add("Grade updated: " + g.getGradeId() + " → " + g.getValue());
    }
}

// Wiring it together (via Facade / GradeService):
Grade grade = new Grade("g01", "s001", "CS101");
grade.addObserver(student); // Student listens
grade.addObserver(admin); // Admin listens
grade.assign(85.0, "t01"); // Both notified automatically

```

Summary & Comparison

The three patterns selected cover all three GoF categories and directly address the core architectural challenges in the Student Information System. Each pattern solves a real problem visible in the project's use cases, diagrams, and feature list.

Pattern	Category	Problem it solves	SIS classes	Design principle
Singleton	Creational	Multiple actors need one shared report engine — duplicates cause inconsistent data	ReportManager, Report, Student, Admin, Course	Single Responsibility, Dependency Inversion
Facade	Structural	Actors would need direct access to 4 subsystems — tight coupling, fragile design	StudentSystemFacade, AuthService, CourseService, GradeService, ReportService	Law of Demeter, Open/Closed
Observer	Behavioral	Grade assignment must notify Student & Admin without coupling Teacher to them	Grade, GradeObserver, Student, Admin, Teacher, Course	Open/Closed, Single Responsibility

Together these three patterns make the SIS modular, extensible, and maintainable. The **Singleton** guarantees a single consistent report engine used across all actors. The **Facade** gives each actor a clean, role-appropriate interface while hiding four internal subsystems. The **Observer** decouples grade state changes from their downstream effects — matching the separation of concerns shown in the project's own swimlane diagrams. No actor is tightly coupled to another, and each class has a single, clear responsibility.